Graph-CPO: Graph-Grounded Contrastive Pedagogical Optimization for Socratic Question Generation

Anonymous Author(s)

Abstract

Math word problems (MWPs) benefit from pedagogical scaffolding through Socratic questioning, yet automating effective question sequences remains challenging. Existing approaches treat question generation as static text-to-text tasks, ignoring the temporal dependencies among mathematical operations and intermediate reasoning steps. We address this by formalizing Socratic questioning as a finite-horizon sequential decision process, where each sub-question targets a specific reasoning dependency. We propose ***Graph-CPO*** *(**G**raph-grounded **C**ontrastive **P**edagogical **O**ptimization)*, a reinforcement learning framework that conditions question generation on a Semantic Dependency Graph constructed from the problem structure. By employing dynamic reward weighting with independently normalized dimensions, Graph-CPO prevents signal domination by trivial objectives while guiding the policy toward logically coherent question sequences. Extensive evaluation demonstrates that Graph-CPO generates superior sub-questions that significantly improve automated solver accuracy (+8.52 points average improvement over the same backbone) and achieves a 95.6% sequence validity rate, competitive with systems using models 20× larger. Human evaluation confirms higher coherence and pedagogical alignment compared to baselines.

CCS Concepts

• **Computing methodologies** → **Natural language generation**; **Reinforcement learning**; • **Applied computing** → *Education*.

Keywords

Graph-CPO, Semantic Dependency Graph, Socratic Question Generation, Reinforcement Learning for Education, Math Word Problem Solving.

ACM Reference Format:

Anonymous Author(s). 2026. Graph-CPO: Graph-Grounded Contrastive Pedagogical Optimization for Socratic Question Generation. In *Proceedings of Proceedings of the ACM Web Conference 2027 (WWW '27).* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Socratic questioning guides problem solving through a sequence of purposeful inquiries [19]. For math word problems (MWPs), automating this process is difficult because useful questions must be fluent, mathematically valid, and aligned with the problem's latent reasoning structure. Prior work suggests that well-designed question sequences can improve both human and automated reasoning [4]. However, most question generation (QG) methods treat generation as a static text-to-text task, which often ignores the temporal dependencies among quantities, operations, and intermediate variables [22, 28]. Consequently, generated questions may be redundant, weakly ordered, or disconnected from the final solution.

We argue that effective Socratic sub-question generation for MWPs requires two properties: **targetedness**, where each question addresses the most informative unresolved reasoning step, and **purpose-orientation**, where the sequence progressively advances the solver toward the final answer. To capture these properties, we formulate Socratic sub-question generation as a finite-horizon sequential decision process. At each timestep, the policy observes the problem, previous questions, and a graph-based reasoning state, then selects the next question as an action that should resolve a specific dependency in the solution path.

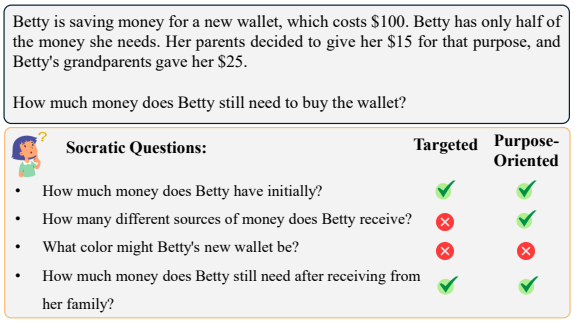


Figure 1: Socratic questioning as a sequential decision process over an MWP: at each timestep the policy observes the problem, the previously asked sub-questions, and the current reasoning state, and targets the next unresolved reasoning dependency in the solution path.

Optimizing such a policy is challenging because pedagogical quality depends on heterogeneous objectives, including relevance, logical coherence, and structural validity. Directly aggregating these rewards can cause *reward collapse*: as training progresses, easy-to-optimize objectives such as format adherence saturate and dominate the aggregate learning signal, so that apparent progress in the scalarized reward masks stagnation or degradation of the harder reasoning-oriented rewards [24, 30]. To address this, we propose **Graph-CPO**, a graph-grounded reinforcement learning framework for Socratic question generation. Graph-CPO first constructs a Semantic Dependency Graph for each MWP, then maps frontier nodes in the graph to graph-conditioned question plans. During optimization, it uses a contrastive relevance reward to align questions with the current target node while penalizing repetition of resolved

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

WWW '27, Dublin, Ireland

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

nodes. It further applies independently normalized reward dimensions and curriculum-based weighting to balance format, relevance, and coherence throughout training.

Our contributions are as follows:

(1) We formulate Socratic sub-question generation for MWPs as a finite-horizon sequential decision process grounded in a Semantic Dependency Graph, and propose Graph-CPO, a multi-objective reinforcement learning approach that combines graph-grounded contrastive rewards with independently normalized, curriculum-weighted reward dimensions to enable targeted and purpose-oriented Socratic questioning.
(2) We investigate the following research questions:
   - **(RQ1):** *Does incorporating Socratic sub-questions improve solution accuracy in math word problems?*
   - **(RQ2):** *Which structural and sequential properties are essential for effective Socratic questioning under a decision-theoretic framework?*
(3) We provide a comprehensive empirical evaluation and ablation analysis showing that our framework generates question sequences with higher coherence and precise target alignment, which significantly improve both automated solver accuracy (+8.52 points on average) and subjective human quality ratings.

2 Related Work

Socratic questioning is a cornerstone of educational pedagogy [34]; in mathematics education it appears as funneling interactions that guide students toward key concepts [35]. LLM-based systems now simulate Socratic dialogue to strengthen mathematical reasoning, from prompt-based tutors [2, 9, 36] to ideators that decompose problems into sequenced sub-questions [21], alongside studies of AI versus human tutors and of reviving Socratic methods in distance education [12, 32].

Educational question generation (QG) with LLMs spans Bloom's-taxonomy assessment generation [10, 28], multiple-choice generation [5], controllable prompt patterns [22], and RLHF-driven multimodal systems [26] (see also reviews of neural QG in education [3]). Reinforcement learning further shapes QG through graph-to-sequence and answer-driven generation [6, 14, 33], fine-tuning of LLMs for educational question generation [20], and multi-agent frameworks for math problem generation [11].

Math word problem (MWP) solving with LLMs has advanced through estimation verification and symbolic solvers [27], abstract-reasoning probes without explicit numbers [7], template-driven paraphrasing [18], iterative distractor refinement [37], step-by-step correction [23], and multimodal benchmarks [38], with human-cognition perspectives surveyed in [17]. Group-sampling RL objectives [30] achieve high scores on MATH and GSM8K, but they optimize single-reward trajectories—precisely the setting that lacks the robustness needed for the multi-objective optimization behind pedagogical scaffolding.

Beyond task-specific reward design, aggregating heterogeneous rewards into a single scalar is a long-standing challenge in RL: naive scalarization lets easy-to-saturate or high-variance dimensions dominate the learning signal [13, 31]. Existing remedies include stabilizing value functions across reward scales (PopArt [16]) and normalizing each reward dimension separately inside policy-gradient updates [24]. In language-model alignment, *reward hacking* has been extensively documented [13, 31]: the agent maximizes a proxy objective while degrading the true objective. GRPO-style objectives [30] normalize rewards within a sampled group, but typically scalarize multiple dimensions *before* normalization—the exact setting in which easy objectives mask harder ones. Graph-CPO instead normalizes each pedagogical dimension independently and only then combines them under a curriculum; we show empirically that this prevents format adherence from masking relevance and coherence during training (Appendix Figure 4).

Our work redefines Socratic questioning as a finite-horizon trajectory over an explicit reasoning structure and contributes a multi-objective optimization scheme for navigating it: a graph-grounded contrastive reward makes each question **targeted**, while independently normalized, curriculum-weighted reward dimensions keep the training signal balanced so that the sequence remains **purpose-oriented**, preventing easy objectives from dominating optimization.

3 Methodology

In this section, we describe Graph-CPO as a structured, three-phase pipeline where each phase produces essential components that the subsequent phase requires. Figure 2 depicts our overall approach. Particularly, **Phase 1 - Semantic Dependency Graph Construction and Question Planning** (Sections 3.1 and 3.2): we construct a Semantic Dependency Graph ($\mathcal{G}$) that transforms unstructured problem text into an explicit topological representation of reasoning dependencies, and convert the current graph state into a compact, graph-conditioned question plan. The graph serves as the structural foundation that provides the state representation required for all subsequent components. **Phase 2 - Targeted Supervised Learning** (Section 3.3): building directly on the structured graph from Phase 1, we train a Graph-Augmented Policy that conditions question generation on specific nodes in $\mathcal{G}$. This step produces a warm-started policy π_θ that mitigates cold-start instabilities in reinforcement learning while ensuring graph-aware question generation. **Phase 3 - Graph-CPO Optimization** (Section 3.4): we integrate the graph structure and the initialized policy through the core Graph-CPO algorithm, which optimizes the question generator by decoupling reward dimensions and applying dynamic weighting strategies. By anchoring the multi-objective optimization to the topological roadmap from Phase 1 and the stable initialization from Phase 2, Graph-CPO effectively bridges local reasoning steps with global problem-solving objectives while preventing reward collapse.

3.1 Semantic Dependency Graph for Targeted Sub-Question Generation

Given an MWP P, we construct a directed acyclic **Semantic Dependency Graph** $\mathcal{G} = (V, E, \tau, \rho, w)$, where each node $v \in V$ is a typed reasoning unit, each edge $e = (u, v) \in E$ is a typed dependency, $\tau(\cdot)$ and $\rho(\cdot)$ denote node and edge types, and $w_e \in [0, 1]$ is the

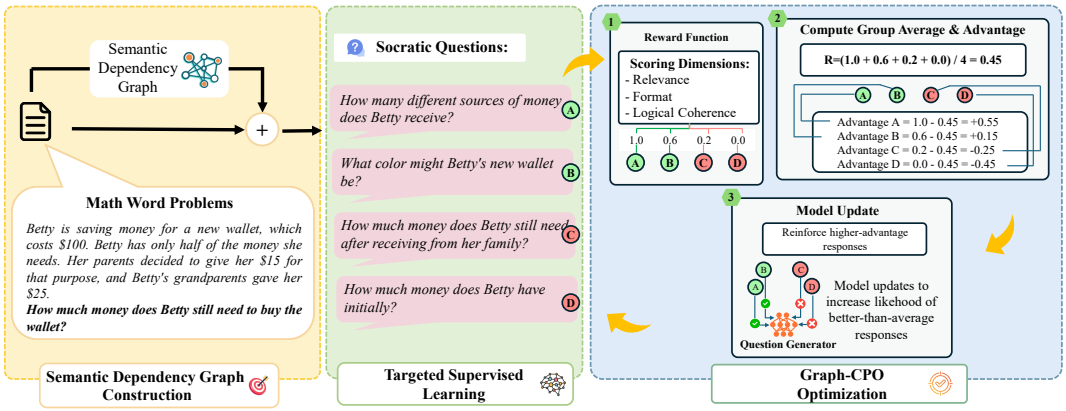


Figure 2: Overall processing pipeline of the proposed framework integrating Graph-CPO. It utilizes the Semantic Dependency Graph to identify the reasoning frontier for targeted question generation. Then, a warm-start strategy is set for reinforcement learning via targeted supervised learning. Subsequently, Graph-CPO optimizes sub-question generation via adaptive reward modeling and curriculum-based learning.

Table 1: Schema of the Semantic Dependency Graph.

Type	Definition	Example / Direction
Node types		
QUANTITY	Numerical value with unit or owner	"12 apples"
ENTITY	Person, object, group, or unit	"Tom", "apples"
VARIABLE	Unknown or intermediate quantity	apples left
OPERATION	Arithmetic/reasoning operator	+, −, ×, ÷, ratio
CONDITION	Constraint or comparison	"twice", "after"
GOAL	Final asked quantity	"how many remain?"
Edge types		
QUANTITY-OF	Quantity belongs to an entity	$q \to e$
OP-INPUT	Node is input to an operation	$x \to op$
DERIVE	Operation produces a derived value	$op \to x$
CONDITION	Node participates in a constraint	$x \to c$
PREREQUISITE	Source must be resolved first	$u \to v$
GOAL-DEP.	Node is required for final goal	$u \to g$

relation confidence. Unlike a generic syntactic parse, $\mathcal{G}$ represents a pedagogical solution topology: an edge $u \to v$ indicates that u should be identified or reasoned about before asking a question targeting v.

Graph construction. The graph is built in three stages. First, a fine-tuned BERT sequence tagger extracts text spans and assigns them to the node types in Table 1. Explicit quantities are normalized with their units, while operation and condition nodes are created from arithmetic cues such as "left", "each", "twice", or "more than". Second, candidate node pairs are formed when they occur in the same sentence, share an entity/unit, are linked by an arithmetic cue, or connect to the final goal. A biaffine relation classifier then predicts either one edge type in Table 1 or NULL. Edges are oriented from prerequisite facts toward derived variables and the goal. If the predicted graph contains a cycle, we remove the lowest-confidence edge in the cycle until a valid Directed Acyclic Graph is obtained. The tagger and the biaffine classifier are trained on silver labels distilled from the solution steps of 12,000 GSM8K/MathQA training problems; the 1,000 expert-annotated graphs used for evaluation (Section 4.2.1) are held out and never used for training. Third, we encode $\mathcal{G}$ with a relation-aware GNN. Each node is initialized by concatenating its span embedding, type embedding, and numeric features:

$$\mathbf{h}_i^{(0)} = \mathrm{BERT}(v_i) \oplus \mathbf{e}_{\tau(v_i)} \oplus \phi_{\mathrm{num}}(v_i)$$

The GNN updates node representations by relation type:

$$\mathbf{h}_i^{(\ell+1)} = \sigma\left(W_0 \mathbf{h}_i^{(\ell)} + \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \frac{1}{c_{i,r}} W_r \mathbf{h}_j^{(\ell)} \right),$$

where $\mathcal{N}_r(i)$ denotes neighbors connected to i by relation r. The final node embedding $\mathbf{h}_i^{(L)}$ is used by the generator and by the graph-grounded relevance reward.

Frontier tracking and node resolution. At timestep t, the graph state is $\mathcal{G}_t = (\mathcal{G}, V_{\mathrm{res}}^t, V_{\mathrm{unres}}^t)$, where "resolved" means that the corresponding reasoning dependency has already been covered by a generated sub-question. Initially, $V_{\mathrm{res}}^0 = \emptyset$ and $V_{\mathrm{unres}}^0 = V$. The targetable reasoning frontier is

$$\mathcal{F}_t = \{ v \in V_{\mathrm{unres}}^t \cap V_Q \mid \mathrm{Pre}(v) \subseteq V_{\mathrm{res}}^t \}, \tag{1}$$

where V_Q excludes purely entity nodes and $\mathrm{Pre}(v)$ denotes the prerequisite nodes of v. Among frontier nodes, the next target v_t is selected by the scoring rule of Section 3.2, which prioritizes nodes closer to the goal, with larger prerequisite depth and higher relation confidence.

After generation, v_t is marked as resolved only if the question is both semantically aligned with the target node and logically addresses its dependency:

$$s_{\mathrm{sim}} = \cos(\mathrm{Enc}(q_t), \mathbf{h}_{v_t}^{(L)}) \geq \tau_{\mathrm{sim}},$$

$$s_{\mathrm{nli}} = P_{\mathrm{NLI}}(\mathrm{entail} \mid [P; \hat{d}(q_t)], d(v_t)) \geq \tau_{\mathrm{nli}},$$

where $\hat{d}(q_t)$ is the declarative target extracted from the generated question and $d(v_t)$ is the verbalized target dependency. If either test fails, the node remains unresolved and the question receives a lower relevance/coherence reward during Graph-CPO optimization. This frontier-based update ensures that generated questions follow the graph topology rather than repeatedly querying already covered

or downstream reasoning steps. Having established how we construct the Semantic Dependency Graph that encodes mathematical reasoning structure, we now describe how this graph guides the generation of targeted Socratic questions through a graph-to-question planning mechanism.

3.2 Graph-to-Question Planning

The Semantic Dependency Graph is not passed to the generator as an unstructured object. Instead, at each timestep we convert the current graph state $\mathcal{G}_t$ (Section 3.1) into a compact **Pedagogical Action Frame** that specifies exactly which reasoning dependency the next Socratic question should address.

Frontier node selection. Among the frontier nodes $\mathcal{F}_t$ of Equation (1), we select the next target by:

$$v_t = \arg\max_{v \in \mathcal{F}_t} \Big[\lambda_g \frac{1}{1 + \mathrm{dist}(v, g)} + \lambda_d \,\mathrm{depth}(v) + \lambda_c \bar{w}(v) - \lambda_r \,\mathrm{red}(v) \Big], \tag{2}$$

where g is the goal node, $\bar{w}(v)$ is the average confidence of edges incident to v, and $\mathrm{red}(v) = \max_{u \in V^t_{\mathrm{res}}} \cos(\mathbf{h}^{(L)}_v, \mathbf{h}^{(L)}_u)$ penalizes revisiting already resolved reasoning units. If $V^t_{\mathrm{res}} = \emptyset$, the redundancy term is set to zero.

Pedagogical action frame. The selected node v_t is converted into an action frame:

$$z_t = \big(d(v_t), \tau(v_t), \iota(v_t), d(\mathrm{Pre}(v_t)), d(\mathcal{N}_1(v_t)), d(V^t_{\mathrm{res}})\big), \tag{3}$$

where $d(\cdot)$ verbalizes node spans, units, and edge labels; $\tau(v_t)$ is the node type; $\mathcal{N}_1(v_t)$ is the one-hop neighborhood; and $\iota(v_t)$ is the pedagogical intent derived from the node type, e.g., IDENTIFY-QUANTITY, INTERPRET-CONDITION, SELECT-OPERATION, or DERIVE-VALUE. The frame is serialized as a short control prompt:

> **Target:** $d(v_t)$; **Intent:** $\iota(v_t)$; **Prerequisites:** $d(\mathrm{Pre}(v_t))$; **Local relation:** $d(\mathcal{N}_1(v_t))$; **Avoid:** $d(V^t_{\mathrm{res}})$. Ask exactly one Socratic question that helps derive the target without directly revealing the answer.

Generator input. For Qwen3-32B, the actual generation input is:

$$x_t = [P;\ q_{1:t-1};\ \mathrm{Ser}(z_t);\ \texttt{<question>}], \tag{4}$$

where P is the MWP text, $q_{1:t-1}$ are previous sub-questions, and $\mathrm{Ser}(z_t)$ is the serialized action frame. To incorporate continuous graph information, we additionally project the target-node and neighborhood embeddings into a graph prefix:

$$\mathbf{p}_t = W_p \left[\mathbf{h}^{(L)}_{v_t} \oplus \mathrm{Pool}_{u \in \mathcal{N}_1(v_t)} \mathbf{h}^{(L)}_u \right] \tag{5}$$

The policy then samples:

$$q_t \sim \pi_\theta(\cdot \mid x_t, \mathbf{p}_t). \tag{6}$$

Thus, the generator receives both a textual plan that is readable by the LLM and a graph embedding that preserves structural information from the GNN.

Binding questions to graph targets. Each generated question is explicitly associated with its target node v_t. Resolved nodes are included in the *Avoid* field and are also penalized by the contrastive relevance reward, preventing the model from repeatedly asking about already covered dependencies. After generation, v_t is marked as resolved only when the question passes the semantic-similarity and NLI checks of Section 3.1; otherwise the graph state is unchanged and the question receives a lower reward during Graph-CPO optimization. This mechanism ensures that question generation follows the dependency topology of $\mathcal{G}$ rather than relying on unconstrained prompting.

3.3 Warm-start Policy Initialization with Targeted Supervised Learning

To provide a stable starting point for the Graph-CPO optimization phase, we first initialize the policy π_θ via targeted supervised learning. This warm-start stabilizes subsequent reinforcement learning by constraining early policy updates to the manifold of syntactically valid and semantically meaningful questions, thereby reducing gradient variance and preventing policy collapse under sparse, long-horizon reward signals [8]. We employ a Graph-Augmented Policy, which enhances a standard transformer encoder-decoder by fusing the problem text embeddings with structure-aware graph representations. This allows the decoder to attend not just to previous tokens, but to the topological state of the reasoning frontier. During this supervised warm-start phase, the policy is trained to imitate ground-truth Socratic sub-question sequences using a standard negative log-likelihood objective. These teacher sequences are constructed from the annotated solution steps of 8,000 training MWPs (4,000 GSM8K and 4,000 MathQA): each solution step is verbalized as one Socratic sub-question aligned to the corresponding graph node, yielding one teacher trajectory per problem. The loss function for this training is defined as:

$$\mathcal{L}_{QG} = -\sum_{i=1}^{n} \log P_{\mathrm{Dec}}(q_i \mid q_{1:i-1}; \mathrm{Enc}(P)), \tag{7}$$

where $P_{\mathrm{Dec}}(\cdot)$ denotes the decoder-defined conditional probability of generating the next sub-question under the current policy, parameterized by the transformer decoder. This autoregressive decoding ensures that the questions build upon each other logically, mirroring the sequential nature of mathematical reasoning.

3.4 Graph-CPO: Graph-Grounded Contrastive Pedagogical Optimization

Supervised warm-starting makes the generator fluent, but it does not directly optimize long-horizon pedagogical utility. We therefore introduce **Graph-CPO**, a GRPO-style policy optimization method [30] that differs from scalar GRPO in two ways: rewards are first normalized independently by pedagogical dimension, and then combined through a curriculum-based weighting schedule. This prevents easy objectives, such as format adherence, from dominating harder graph-grounded reasoning signals. (Here, CPO denotes *Contrastive Pedagogical Optimization*; it is unrelated to Constrained Policy Optimization [1].)

Graph-conditioned supervised anchor. Given the action frame z_t and graph prefix $\mathbf{p}_t$ from Section 3.2, we retain a supervised anchoring loss:

$$\mathcal{L}_{target} = -\sum_{t=1}^{H} \log \pi_\theta \left(q^*_t \mid P, q^*_{<t}, \mathrm{Ser}(z_t), \mathbf{p}_t\right), \tag{8}$$

Table 2: Pedagogical role of the three reward dimensions.

Reward	Property	Role
R_{rel} Contrastive relevance	Targetedness / non-redundancy	Aligns the question with the current frontier node while penalizing similarity to already resolved nodes.
R_{coh} Logical coherence	Sequential progression	Checks whether the question is consistent with prerequisites and advances the target dependency.
R_{form} Format adherence	Generation validity	Ensures parseable output with the required question format for RL training.

where q_t^* is the teacher or gold sub-question aligned to target node v_t. This term preserves linguistic fluency and keeps generation tied to the graph-to-question plan.

Reward modeling. For an action $a_t = q_t$ targeting node v_t, we define three reward dimensions: $K = \{\text{rel}, \text{coh}, \text{form}\}$. Table 2 summarizes their pedagogical roles.

Contrastive relevance. Let $\mathbf{e}_t = \text{Enc}(q_t)$ and $\mathbf{h}_{v_t}^{(L)}$ be the GNN embedding of the target node. Relevance rewards alignment to the frontier node while penalizing regression to resolved nodes:

$$R_{\text{rel}}(q_t, v_t) = \cos(\mathbf{e}_t, \mathbf{h}_{v_t}^{(L)}) - \eta\Delta_t, \tag{9}$$

where

$$\Delta_t = \begin{cases} \max\limits_{u \in V_{\text{res}}^t} \cos(\mathbf{e}_t, \mathbf{h}_u^{(L)}), & V_{\text{res}}^t \neq \emptyset, \\ 0, & V_{\text{res}}^t = \emptyset. \end{cases} \tag{10}$$

Logical coherence. Instead of applying NLI directly between two raw questions, we evaluate whether the generated question logically addresses the target dependency. Let $\hat{d}(q_t)$ denote the declarative intent of the question, e.g., "the question asks for the remaining apples", let $d(v_t)$ verbalize the target node, and let $\psi_t = [P; d(\text{Pre}(v_t)); \hat{d}(q_t)]$ denote the evidence available to the judge. We define:

$$R_{\text{coh}}(q_t, v_t) = P_{\text{NLI}}(\text{entail} \mid \psi_t, d(v_t)) - P_{\text{NLI}}(\text{contradict} \mid \psi_t, d(v_t)). \tag{11}$$

We use DeBERTa-v3-large-mnli as the NLI judge, which is architecturally distinct from the Qwen3-32B generator, reducing self-rewarding bias.

Format adherence. Finally, R_{form} is a binary reward:

$$R_{\text{form}}(q_t) = \mathbf{1}\{q_t \text{ has valid tags}\}. \tag{12}$$

This strict binary reward enforces valid XML tagging of the generated output, which is necessary for stable parsing during RL training, and is intentionally down-weighted as training progresses.

Decoupled adaptive advantage. For each prompt, we sample M rollouts. For reward dimension k, we compute a group-normalized advantage before aggregation:

$$A_{m,k,t} = \frac{R_{m,k,t} - \mu_{k,t}}{\sigma_{k,t} + \epsilon}, \tag{13}$$

where $\mu_{k,t}$ and $\sigma_{k,t}$ are computed over the M sampled rollouts. The composite advantage is:

$$\hat{A}_{m,t} = \text{Norm}_m\left(\sum_{k \in K} \omega_k(u) A_{m,k,t}\right), \tag{14}$$

where u is the training update step. We use a curriculum schedule:

$$\begin{aligned} \omega_{\text{form}}(u) &= \omega_{\min} + (\omega_{\max} - \omega_{\min})e^{-\lambda u}, \\ \omega_{\text{rel}}(u) &= \omega_{\text{coh}}(u) = \frac{1 - \omega_{\text{form}}(u)}{2}. \end{aligned} \tag{15}$$

Thus, early training emphasizes valid structure, while later training shifts weight toward relevance and logical coherence.

Graph-CPO objective. During each RL update, rollouts are sampled from the old policy $\pi_{\theta_{\text{old}}}$, while a fixed reference policy π_{ref} is used only for KL regularization. The probability ratio is therefore:

$$r_{m,t}(\theta) = \frac{\pi_\theta(a_t^m \mid s_t^m)}{\pi_{\theta_{\text{old}}}(a_t^m \mid s_t^m)}. \tag{16}$$

With the clipped ratio $\bar{r}_{m,t}(\theta) = \text{clip}(r_{m,t}(\theta), 1 - \epsilon_c, 1 + \epsilon_c)$, the minimized Graph-CPO loss is:

$$\begin{aligned} \mathcal{L}_{\text{CPO}} = -\mathbb{E}_{m,t}\left[\min\left(r_{m,t}(\theta)\hat{A}_{m,t}, \bar{r}_{m,t}(\theta)\hat{A}_{m,t}\right)\right] \\ + \beta\mathbb{E}_{m,t}\left[D_{\text{KL}}\left(\pi_\theta(\cdot \mid s_t^m) \| \pi_{\text{ref}}(\cdot \mid s_t^m)\right)\right]. \end{aligned} \tag{17}$$

This notation separates the old sampling policy from the reference policy, following PPO/GRPO-style optimization [29, 30].

Overall objective. The final loss combines supervised anchoring and Graph-CPO:

$$\mathcal{L}_{\text{Graph-CPO}} = \alpha(u)\mathcal{L}_{target} + (1 - \alpha(u))\mathcal{L}_{\text{CPO}}, \tag{18}$$

where $\alpha(u) = \alpha_{\min} + (\alpha_0 - \alpha_{\min})e^{-\lambda_\alpha u}$ decays monotonically from α_0 to $\alpha_{\min}$ with training (Appendix Table 10). Early updates remain close to the teacher trajectory, while later updates are increasingly driven by graph-grounded pedagogical rewards.

4 Experiments and Results

4.1 Experiment Setups

4.1.1 Datasets, Implementation, and Baselines. We evaluate Graph-CPO on GSM8K [8], MathQA [25], OlympiadBench [15], and AIME. Unless otherwise stated, all controlled experiments use **Qwen3-32B** as the backbone. Because full-parameter training of a 32B model does not fit into the memory budget of 40 GB GPUs, we train with LoRA (rank 64, scaling 128, dropout 0.05, applied to attention and MLP projection layers; about 190M trainable parameters) together with the 8-bit Adam optimizer. The supervised warm-start runs for 3 epochs; the Graph-CPO RL phase runs for 50 epochs with periodic validation, all on 4 NVIDIA A100-40GB GPUs. We provide full hyperparameters in Appendix Table 10.

We compare against three groups of baselines: **(i)** open-source educational QG frameworks, including SoQG [4], BloomLLM [10], and Automated Educational QG [28]; **(ii)** distilled reasoning models, including DeepSeek-R1-Distill-Llama-8B in a zero-shot setting; and **(iii)** commercial generators, including Gemini-2.5-Pro, GPT-5, Grok-4, and Claude-4.1-Opus. These baselines test both controlled backbone-level improvements and performance relative to stronger external generators.

4.1.2 Computational Cost. A full cost comparison is given in Appendix 6.2 (Table 11). The headline figure: Graph-CPO's overhead over scalar GRPO is modest—636 vs. 610 GPU-h for 50 RL epochs on 4 A100-40GB GPUs, with 5.6s vs. 5.4s average per-problem generation latency.

4.2 Evaluation

4.2.1 Semantic Dependency Graph Construction Quality. We evaluate the SDG construction pipeline against expert-annotated graphs for 1,000 MWPs (500 GSM8K, 500 MathQA). Two mathematics education researchers annotated entities, relations, and dependency orderings, with substantial agreement (κ = 0.85/0.81/0.88 for entity boundaries, relation types, and topological order); disagreements were adjudicated by a third expert. The tagger and relation classifier under evaluation were trained only on silver labels (Section 3.1); these 1,000 annotated graphs are held out from all training.

Table 3: Semantic Dependency Graph Construction Performance.

Evaluation Metric	Performance
Entity Extraction	
Entity Boundary F1 / Type Acc.	88.2% / 91.4%
Quantity / Operation / Variable F1	92.1% / 84.6% / 86.8%
Relation Prediction	
Edge Precision / Recall / F1	85.7% / 81.2% / 83.4%
Relation Type Accuracy	79.6%
End-to-End Graph Quality	
Critical Path Preservation	94.3%
Normalized Graph Edit Distance	0.142
Topological Correctness	89.7%

As shown in Table 3, the SDG pipeline preserves most solution-critical dependencies, with 94.3% Critical Path Preservation and 89.7% Topological Correctness. Errors mainly come from implicit operation recognition (42%), complex multi-hop dependencies (31%), and ambiguous entity references (27%). Graph quality is strongly correlated with downstream accuracy: problems with above-95% Critical Path Preservation achieve 91.2% accuracy, compared with 84.7% for 80–95% preservation ($r = 0.78$, $p < 0.001$).

4.2.2 GNN and Graph Representation. To verify that the improvement is not merely due to graph prompting, we ablate how the graph is provided to the generator. Targeted measures whether the generated question is aligned with the selected frontier node, Coherence is the NLI-based sequential consistency score, and Acc. denotes MathQA accuracy under the direct-augmentation protocol (Protocol A, defined below).

Table 4 shows that textual graph serialization alone substantially improves over the no-graph baseline, confirming that explicit dependency structure is useful for Socratic planning. However, node embeddings without message passing and text-only graph prompts remain below the full relation-aware GNN model. The full graph encoder improves MathQA accuracy from 82.37 to 89.60 over node-only embeddings, indicating that edge-typed message passing captures multi-hop prerequisite relations that are not available from isolated node representations. The oracle-graph upper bound further shows that remaining errors partly originate from imperfect graph construction, rather than from the Graph-CPO optimization itself.

4.2.3 ***Overall performance.*** Our study evaluates (i) the intrinsic quality of generated Socratic sub-question sequences and (ii) their downstream utility for training. Our primary comparisons focus on **Qwen3-32B-based generators** to isolate the impact of optimization on a shared backbone (Qwen3-32B and Graph-CPO). We additionally report large open-source and commercial generators as a practical performance reference.

Table 4: Ablation of graph representation and GNN encoder on MathQA (Protocol A: direct augmentation).

Variant	Graph repr.	GNN	Tgt.	Coh.	Acc.
No graph	None	–	58.7	72.4	56.45
Text graph only	Serialized $\mathcal{G}$	–	78.9	83.6	79.84
Node emb. only	BERT node emb.	–	81.5	85.2	82.37
Full graph	Ser. $\mathcal{G}$ + GNN	✓	**88.6**	**91.7**	**89.60**
Oracle graph	Expert $\mathcal{G}$ + GNN	✓	91.8	93.4	91.26

Evaluation Protocol. We separate two protocols that answer different questions.

Protocol A (direct augmentation): intrinsic quality and in-context utility. For each benchmark problem, the generator first produces a sequence of Socratic sub-questions, which are provided as context when Qwen3-32B (or DeepSeek-R1-Distill-Llama-8B) solves the problem under fixed constraints (4,096-token limit, 300-second timeout). We report three quality measures: *Success Rate* (the percentage of augmented MWPs solved correctly, requiring correctness of both the sub-questions and the final answer; used to filter valid augmented pairs), the *Purpose-Orientation* score (NLI-based sequential coherence), and the *Targeted* score (BERTScore overlap with MWP entities/operators). Controls include (i) original MWPs without sub-questions and (ii) random/non-Socratic sub-questions. All ablation studies and both research questions are analyzed under Protocol A.

Protocol B (downstream fine-tuning): transferable utility. We sample 1,000 seed MWPs from the GSM8K training set and generate 3–5 Socratic sub-question variants per seed (3,000–5,000 total). For each generator, we fine-tune a fresh Qwen3-32B solver with the same LoRA configuration on the valid augmented QA pairs produced in the previous stage, using the generated sub-questions as chain-of-thought context; the baseline is fine-tuned on the unaugmented training set. Downstream utility is measured as mean accuracy (averaged over 16 decoding runs per problem) on four benchmarks (AIME-24, GSM8K, MathQA, and OlympiadBench). Table 5 reports Protocol B; all other result tables (ablation studies) and the RQ analyses report Protocol A.

Results. Table 5 demonstrates that Graph-CPO, built on the Qwen3-32B backbone, achieves a substantial +8.52 points improvement over the baseline (61.15% vs 52.63%; std 0.9 across the 16 decoding runs, paired t-test $p < 0.001$), significantly outperforming existing open-source question generation frameworks which show only modest gains of +0.87 to +1.85 points. The improvements are most pronounced on challenging multi-step reasoning benchmarks, with notable gains on Olympiad (+14.0 points) and AIME-24 (+8.4 points). Remarkably, on GSM8K, Graph-CPO reaches 96.2%, surpassing even advanced commercial generators including GPT-5 and Claude-4.1-Opus. While commercial systems achieve higher overall

Table 5: Comparison under Protocol B (downstream fine-tuning). Superscripts denote point changes vs. the Qwen3-32B generator baseline. Non-Qwen3-32B generators are included for reference and may reflect both generator design and backbone.

	Benchmark Datasets				Avg.
	AIME-24	**GSM8K**	**MathQA**	**Olympiad**	
Open-Sourced Generators Frameworks					
Qwen3-32B	31.0	93.4	61.6	24.5	52.63
SoQG	32.5 +1.5	94.1 +0.7	62.8 +1.2	24.6 +0.1	53.50 +0.87
BloomLLM	33.2 +2.2	94.6 +1.2	63.5 +1.9	25.1 +0.6	54.10 +1.47
Automated Educational QG	34.5 +3.5	94.8 +1.4	63.2 +1.6	25.4 +0.9	54.48 +1.85
Advanced Commercial Generators					
Gemini-2.5-Pro	40.1 +9.1	94.4 +1.0	71.8 +10.2	47.5 +23.0	63.45 +10.82
GPT-5	42.5 +11.5	95.8 +2.4	72.6 +11.0	48.2 +23.7	64.78 +12.15
Grok-4	41.8 +10.8	95.2 +1.8	72.1 +10.5	47.9 +23.4	64.25 +11.62
Claude-4.1-Opus	43.1 +12.1	96.1 +2.7	73.4 +11.8	48.9 +24.4	65.38 +12.75
Graph-CPO (Qwen3-32B backbone)	**39.4** +8.4	**96.2** +2.8	**70.5** +8.9	**38.5** +14.0	**61.15** +8.52

Table 6: MathQA accuracy (%) with and without Graph-CPO sub-questions (Protocol A). Superscripts denote the absolute gain from adding sub-questions.

Variation	Qwen3-32B	DeepSeek-R1-8B
MWP *w/* sub-questions	**89.60** +33.15	**76.63** +27.17
MWP *w/o* sub-questions	56.45	49.46

averages, our method closes approximately 67% of the performance gap between the open-source baseline and the best commercial system, demonstrating that graph-grounded pedagogical scaffolding can effectively compensate for scale disadvantages in mathematical reasoning tasks.

4.2.4 *RQ1: Does incorporating Socratic sub-questions improve solution accuracy in math word problems?*

Table 6 shows that adding Graph-CPO sub-questions substantially improves MathQA accuracy across solvers under Protocol A. With Qwen3-32B, accuracy increases from 56.45% to 89.60% (+33.15; mean±std over 16 decoding runs: 89.60 ± 0.60 vs. 56.45 ± 0.91, paired t-test $p < 0.001$), and with DeepSeek-R1-Distill-Llama-8B from 49.46% to 76.63% (+27.17, $p < 0.001$). This indicates that structured sub-questions provide useful intermediate reasoning anchors rather than merely adding longer context.

Table 7 shows that the gains depend on three properties: coverage, order, and relevance. Removing sub-questions monotonically reduces accuracy, shuffling them causes an 8.66-point drop, and replacing them with generic baseline questions performs even worse than no scaffolding. Thus, effective Socratic guidance must be sufficiently granular, dependency-aware, and problem-specific.

4.2.5 *RQ2: Which structural and sequential properties are essential for effective Socratic questioning under a decision-theoretic framework?*

Targeted question generation. Table 8 evaluates how graph structure affects reasoning quality. Increasing graph path density consistently reduces reasoning errors and improves downstream accuracy. The full SDG reduces the reasoning error rate from 24.5% to 1.2% and improves MathQA accuracy from 56.45% to 89.60%, showing that

Table 7: Controlled ablations of Socratic sub-questions on MathQA (Protocol A). $\{q\}^k$ uses a random fraction k of the sub-questions; shuffle($\{q\}$) permutes question order; ⟨base-ques⟩ uses untuned Qwen3-32B questions. Red markers denote drops relative to the full set $\{q\}^1$.

Variation	Accuracy
Granularity	
MWP *w/* $\{q\}^0$	56.45 -33.15
MWP *w/* $\{q\}^{0.25}$	65.94 -23.66
MWP *w/* $\{q\}^{0.5}$	73.56 -16.04
MWP *w/* $\{q\}^{0.75}$	83.36 -6.24
MWP *w/* $\{q\}^1$	**89.60**
Order	
MWP *w/* shuffle($\{q\}$)	80.94 -8.66
Relevance	
MWP *w/* ⟨base-ques⟩	54.57 -35.03

Table 8: Effect of graph structure on reasoning quality and MathQA accuracy (Protocol A). Path density: fraction of solution-critical (gold) dependencies represented in the planning structure. Error: percentage of generated sub-questions judged by the NLI check to violate the gold dependency order or contradict the problem statement.

Strategy	Path Density	Error (%)	Acc. (%)
No Graph	0%	24.5	56.45
Keyword-based	40%	15.2	68.30
Pruned Graph	65%	8.4	77.12
Full Graph	**100%**	**1.2**	**89.60**

explicit dependency paths help the generator avoid unsupported reasoning jumps.

Purpose orientation. We assess it using *sequence validity*, the percentage of generated question sequences that are logically sound and solvable by a strong teacher model under strict reasoning constraints. As shown in Table 9, Graph-CPO reaches 95.6%, improving over the Qwen3-32B backbone by +6.5 points and approaching much larger open-source and commercial generators. This suggests that graph-grounded optimization improves not only where the

Table 9: Generator sequence validity rates (%).

Generator	Validity (%)	
Qwen3-32B	89.1	
Qwen3-235B-A22B	95.1	+6.0
DeepSeek-v3.1-671B	96.5	+7.4
Gemini-2.5-Pro	94.2	+5.1
GPT-5	95.8	+6.7
Grok-4	95.7	+6.6
Claude-4.1-Opus	96.9	+7.8
Graph-CPO	**95.6**	+6.5

model attends, but also whether the resulting sequence forms a usable reasoning path.

4.2.6 Training Dynamics: Preventing Reward Collapse. Training logs (Figure 4 in Appendix 6.3) contrast the per-dimension training rewards of scalar GRPO and Graph-CPO and show why reward decoupling matters. Both methods saturate the format-adherence reward within roughly the first 100 steps, and in both cases this early saturation coincides with an initial dip in contrastive relevance—the signature of easy objectives crowding out harder ones. Scalar GRPO never recovers: after about 400 steps its relevance score decays and its logical coherence drops sharply, i.e., it maintains surface compliance while reasoning quality degrades (reward hacking). Graph-CPO's relevance rises steadily and its coherence stays stable throughout training, indicating that independently normalized reward dimensions combined with curriculum weighting prevent reward collapse. Detailed training curves and further analysis are provided in Appendix 6.3.

4.3 Human Quality Evaluation

We evaluate generated questions on 400 randomly sampled test MWPs. Four mathematics researchers rated outputs from Graph-CPO and the Qwen3-32B baseline on a 5-point Likert scale, with 20% overlap for reliability (weighted $\kappa = 0.81$). The criteria include repetition, factuality, logical relevance, sequencing, granularity, completeness, and fluency.

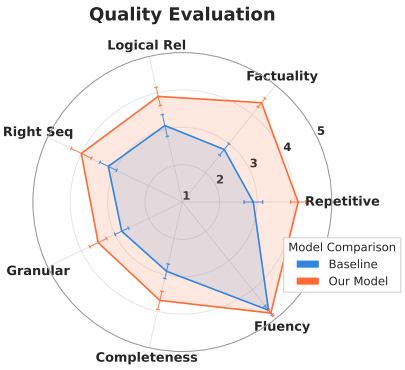


Figure 3: Human evaluation of generated sub-questions: Graph-CPO vs. the Qwen3-32B baseline, showing mean and standard deviation over 400 MWPs rated by four raters.

Figure 3 shows that Graph-CPO consistently outperforms the Qwen3-32B baseline on all non-fluency dimensions. The largest gains appear in factuality (4.3 vs. 3.5) and correct sequencing (4.4 vs. 3.6), indicating that graph-grounded optimization better aligns questions with the mathematical dependencies of the problem. Both methods achieve high fluency, but Graph-CPO yields more stable scores across structural criteria such as completeness, granularity, and logical relevance.

Qualitative case study. A worked example comparing Graph-CPO's generated question sequence with the baseline for a representative MWP is given in Appendix 6.4 (Table 15).

Ethical considerations. The human study involved four mathematics researchers who voluntarily participated and consented to rate anonymized question outputs; no student or personal data were used. The generated sub-questions are intended as supplemental scaffolds rather than as answer keys, and downstream deployments should pair them with answer verification to avoid leaking solutions to learners.

5 Conclusion

We introduced a novel framework for generating effective Socratic sub-questions for Math Word Problems by modeling the process as a finite-horizon sequential decision task anchored to a Semantic Dependency Graph. This structural grounding reduces reasoning error rates from 24.5% to 1.2% and achieves a 95.6% sequence validity rate through decoupled reward normalization and curriculum-based weighting that prevent reward collapse. Beyond automated math tutoring, our work demonstrates that explicitly modeling task structure through dependency graphs can substantially enhance reinforcement learning for generation tasks with complex compositional requirements. The success of treating heterogeneous reward signals as independently scaled dimensions rather than scalar aggregates suggests broader applicability to multi-objective optimization in language generation. Future work should explore cross-domain generalization to other structured reasoning tasks and investigate personalization strategies that adapt question sequencing to individual learner knowledge states.

References

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. 2017. Constrained policy optimization. In *International Conference on Machine Learning*. PMLR, 22–31.

[2] Tosin Adewumi, Foteini Simistira Liwicki, Marcus Liwicki, Viktor Gardelli, Lama Alkhaled, and Hamam Mokayed. 2026. Findings of Mega: Math explanation with LLMs using the socratic method for active learning. *IEEE Signal Processing Magazine* 42, 6 (2026), 77–94.

[3] Said Al Faraby, Adiwijaya Adiwijaya, and Ade Romadhony. 2024. Review on neural question generation for education purposes. *International Journal of Artificial Intelligence in Education* 34, 3 (2024), 1008–1045.

[4] Beng Heng Ang, Sujatha Das Gollapalli, and See Kiong Ng. 2023. Socratic question generation: A novel dataset, models, and evaluation. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 147–165.

[5] Giorgio Biancini, Alessio Ferrato, and Carla Limongelli. 2024. Multiple-choice question generation using large language models: Methodology and educator insights. In *Adjunct Proceedings of the 32nd ACM Conference on User Modeling, Adaptation and Personalization*. 584–590.

[6] Yu Chen, Lingfei Wu, and Mohammed J Zaki. 2019. Reinforcement learning based graph-to-sequence model for natural question generation. *arXiv preprint arXiv:1908.04942* (2019).

[7] Ziling Cheng, Meng Cao, Leila Pishdad, Yanshuai Cao, and Jackie CK Cheung. 2025. Can llms reason abstractly over math word problems without cot? disentangling abstract formulation from arithmetic computation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 14317–14344.

[8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).

[9] Yuyang Ding, Hanglei Hu, Jie Zhou, Qin Chen, Bo Jiang, and Liang He. 2024. Boosting large language models with socratic method for conversational mathematics teaching. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 3730–3735.

[10] Nghia Duong-Trung, Xia Wang, and Miloš Kravčík. 2024. Bloomllm: Large language models based question generation combining supervised fine-tuning and bloom's taxonomy. In *European Conference on Technology Enhanced Learning*. Springer, 93–98.

[11] Bisni Fahad Mon, Asma Wasfi, Mohammad Hayajneh, Ahmad Slim, and Najah Abu Ali. 2023. Reinforcement learning in education: A literature review. In *Informatics*, Vol. 10. MDPI, 74.

[12] Hoda Fakour and Moslem Imani. 2025. Socratic wisdom in the age of AI: a comparative study of ChatGPT and human tutors in enhancing critical thinking skills. In *Frontiers in Education*, Vol. 10. Frontiers Media SA, 1528603.

[13] Leo Gao, John Schulman, and John Hilton. 2023. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*. PMLR, 9008–9031.

[14] Menghong Guan, Subrota Kumar Mondal, Hong-Ning Dai, and Haiyong Bao. 2023. Reinforcement learning-driven deep question generation with rich semantics. *Information Processing & Management* 60, 2 (2023), 103232.

[15] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. 2024. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 3828–3850.

[16] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. 2021. Popart: Adaptive value function estimation in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 8465–8474.

[17] Zhenya Huang, Jiayu Liu, Xin Lin, Zhiyuan Ma, Shangzi Xue, Tong Xiao, Qi Liu, Yee Whye Teh, and Enhong Chen. 2025. Foundation of Intelligence: Review of Math Word Problems from Human Cognition Perspective. *arXiv preprint arXiv:2510.21999* (2025).

[18] Xiaoqiang Kang, Zimu Wang, Xiaobo Jin, Wei Wang, Kaizhu Huang, and Qiufeng Wang. 2025. Template-driven llm-paraphrased framework for tabular math word problem generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24303–24311.

[19] Greg Kestin, Kelly Miller, Anna Klales, Timothy Milbourne, and Gregorio Ponti. 2025. AI tutoring outperforms in-class active learning: an RCT introducing a novel research-based design in an authentic educational setting. *Scientific Reports* 15, 1 (2025), 17458.

[20] Salima Lamsiyah, Abdelkader El Mahdaouy, Aria Nourbakhsh, and Christoph Schommer. 2024. Fine-tuning a large language model with reinforcement learning for educational question generation. In *International Conference on Artificial Intelligence in Education*. Springer, 424–438.

[21] Xinping Lei, Tong Zhou, Yubo Chen, Kang Liu, and Jun Zhao. 2025. Motivgraph-soiq: Integrating motivational knowledge graphs and socratic dialogue for enhanced llm ideation. *arXiv preprint arXiv:2509.21978* (2025).

[22] Kunze Li and Yu Zhang. 2024. Planning first, question second: An LLM-guided method for controllable question generation. In *Findings of the Association for Computational Linguistics: ACL 2024*. 4715–4729.

[23] Yiyao Li, Dhanish Musharraf Ubaidali, Lu Wang, and Wenyu Zhang. 2025. Step-by-step correction of llm-based math word problems solutions. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.

[24] Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, et al. 2026. Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization. *arXiv preprint arXiv:2601.05242* (2026).

[25] Rikteshwar Majumder, Deepak Sanyal, Paban Kumar Patra, and Purnendu Ganguly. 2019. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 2357–2367.

[26] Ahmed Mubarak, Amna Ahmed, Amira Nasser, Aya Mohamed, Fares El-Sadek, Mohammed Ahmed, Ahmed Salah, and Youssef Sobhy. 2025. QuesGenie: Intelligent Multimodal Question Generation. *arXiv preprint arXiv:2509.03535* (2025).

[27] Mitchell Piehl, Dillon Wilson, Ananya Kalita, and Jugal Kalita. 2025. Solving Math Word Problems Using Estimation Verification and Equation Generation. In *2025 International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 432–439.

[28] Nicy Scaria, Suma Dharani Chenna, and Deepak Subramani. 2024. Automated educational question generation at different bloom's skill levels using large language models: Strategies and evaluation. In *International conference on artificial intelligence in education*. Springer, 165–179.

[29] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[30] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).

[31] Joar Skalse, Nikolaus H R Howe, Dmitrii Krasheninnikov, and David Krueger. 2022. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, Vol. 35.

[32] Giannis Stamatellos. 2025. Socratic Methodologies and Artificial Intelligence against Disinformation: A Philosophical Approach. *The International Journal of the Platonic Tradition* 19, 1 (2025), 95–102.

[33] Liuyin Wang, Zihan Xu, Zibo Lin, Hai-Tao Zheng, and Ying Shen. 2020. Answer-driven deep question generation based on reinforcement learning. In *Proceedings of the 28th International Conference on Computational Linguistics*. 5159–5170.

[34] David Wood, Jerome S Bruner, and Gail Ross. 1976. The role of tutoring in problem solving. *Journal of child psychology and psychiatry* 17, 2 (1976), 89–100.

[35] Terry Wood. 1994. Patterns of interaction and the culture of mathematics classrooms. In *Cultural perspectives on the mathematics classroom*. Springer, 149–168.

[36] Linjin Xi, Yi Zhang, and Qiyun Wang. 2025. Investigating the effects of an LLM-based Socratic conversational agent on students' academic performance and reflective thinking in higher education. *Computers & Education* (2025), 105494.

[37] Kaiqi Yang, Hang Li, Yucheng Chu, Zitao Liu, Mi Tian, and Hui Liu. 2025. Iterative LLM-Based Generation and Refinement of Distracting Conditions in Math Word Problems. In *2025 IEEE International Conference on Teaching, Assessment, and Learning for Engineering (TALE)*. IEEE, 1–8.

[38] Renrui Zhang, Dongzhi Jiang, Yichi Zhang, Haokun Lin, Ziyu Guo, Pengshuo Qiu, Aojun Zhou, Pan Lu, Kai-Wei Chang, Yu Qiao, et al. 2024. Mathverse: Does your multi-modal llm truly see the diagrams in visual math problems?. In *European Conference on Computer Vision*. Springer, 169–186.

Table 11: Computational cost comparison. GPU-h is total training cost on 4 A100-40GB GPUs: the SFT row is measured over 50 SFT epochs and the RL rows over 50 RL epochs; the 3-epoch warm-start initialization adds about 20 GPU-h. Lat. denotes average generation time per problem.

Method	Time/ep.	GPU-h	Lat. (s)	Mem. (GB)
Qwen3-32B SFT	0.17h	34	2.4	32.1
GRPO scalar	3.05h	610	5.4	38.7
Graph-CPO	3.18h	636	5.6	39.1
Graph-CPO w/o NLI	2.36h	472	3.8	37.8

6 Appendix

6.1 Training Hyperparameters

Table 10: Graph-CPO training hyperparameters.

Parameter	Value
Model Architecture	
Backbone	Qwen3-32B
Fine-tuning	LoRA $r = 64$, scaling 128, attn.+MLP
Trainable parameters	∼190M
GNN layers (L)	3
GNN hidden dimension	768
Graph prefix dimension	1024
Data	
Silver-labeled SDG training problems	12,000
Teacher Socratic sequences (warm-start)	8,000 MWPs
Supervised Warm-Start (Section 3.3)	
Learning rate	2e-5
Batch size	16 per GPU
Gradient accumulation	4 steps
Warm-up steps	500
Epochs	3
Graph-CPO RL Training (Section 3.4)	
Learning rate	1e-5
RL epochs	50
Rollouts per prompt (M)	8
KL coefficient (β)	0.01
Clip threshold (ϵ_c)	0.2
Curriculum decay (λ)	0.01
$\omega_{\min}$ / $\omega_{\max}$	0.1 / 0.8
Anchor schedule (α_0, $\alpha_{\min}$, λ_α)	0.8, 0.05, 0.01
Contrastive penalty (η)	0.5
Similarity threshold (τ_{sim})	0.75
NLI threshold (τ_{nli})	0.6
Frontier Node Selection (Eq. 2)	
λ_g / λ_d / λ_c / λ_r	0.4 / 0.3 / 0.2 / 0.1

6.2 Computational Cost Analysis

Table 11 reports training and inference cost. GPU-hours are computed as wall-clock time multiplied by the number of GPUs.

Graph-CPO is more expensive than SFT because RL requires multiple rollouts and reward computation. However, its overhead over scalar GRPO is small, since dynamic reward normalization and graph-prefix conditioning add little cost. The main extra cost comes from NLI-based coherence scoring: removing NLI reduces training time from 3.18h to 2.36h per epoch and latency from 5.6s to 3.8s. Graph construction is performed offline and cached, while NLI scoring is batched across rollouts.

Table 12: Ablation of Graph-CPO reward dimensions on MathQA. DW denotes dynamic reward weighting. All variants use the same Qwen3-32B backbone and graph-conditioned action frames.

Variant	R_{rel}	R_{coh}	R_{form}	DW	MathQA
SFT only	–	–	–	–	82.73
GRPO scalar reward	✓	✓	✓	–	84.96
w/o relevance	–	✓	✓	✓	78.41
w/o coherence	✓	–	✓	✓	83.62
w/o format	✓	✓	–	✓	86.18
w/o dynamic weighting	✓	✓	✓	–	86.47
Full Graph-CPO	✓	✓	✓	✓	**89.60**

6.3 Training Rewards Comparison

From the training curves of GRPO and Graph-CPO on Qwen3-32B shown in Figure 4, we observe that the Question Generator model initially tends to maximize the less complex reward dimensions regardless of the optimization framework. In this setting, the Format Adherence reward is easier to optimize, leading both GRPO and Graph-CPO to achieve near-perfect format scores within approximately the first 100 training steps. Crucially, this rapid rise in format adherence coincides with an early dip in the Contrastive Relevance reward, indicating an initial competition between the structural and semantic objectives. During this baseline phase, the model prioritizes satisfying surface-level formatting constraints, temporarily compromising the more challenging math-grounded relevance goal.

However, looking at the long-term trajectories, Graph-CPO recovers and enhances the Contrastive Relevance reward significantly more effectively than GRPO, ultimately securing higher relevance scores. This divergence becomes highly pronounced after 400 training steps, where GRPO's training begins to destabilize—evidenced by a gradual decay in its Contrastive Relevance score while Graph-CPO maintains a steady upward trajectory. Furthermore, while both frameworks maintain near-perfect format adherence scores throughout the training horizon, we also track the logical coherence metric within each training batch to evaluate how robustly the policies preserve reasoning quality. The results show a striking contrast: as GRPO approaches the 400-step mark, its logical coherence score begins to degrade sharply, signaling a vulnerability to reward hacking where structural compliance is maintained at the expense of sound reasoning. In contrast, Graph-CPO consistently stabilizes and improves its logical coherence score, demonstrating that our graph-grounded contrastive optimization successfully prevents reward collapse.

Table 12 isolates the contribution of each reward dimension. The SFT-only model reaches 82.73%, showing that graph-conditioned supervised warm-starting already provides useful structural guidance, but remains below the RL-optimized variants. Removing R_{rel} causes the largest degradation, dropping performance to 78.41%, which confirms that explicit alignment with the current frontier node and penalization of resolved nodes are central to targeted Socratic questioning. Removing R_{coh} also substantially reduces accuracy to 83.62%, indicating that sequential consistency is necessary for producing useful multi-step scaffolding. The smaller but clear drop

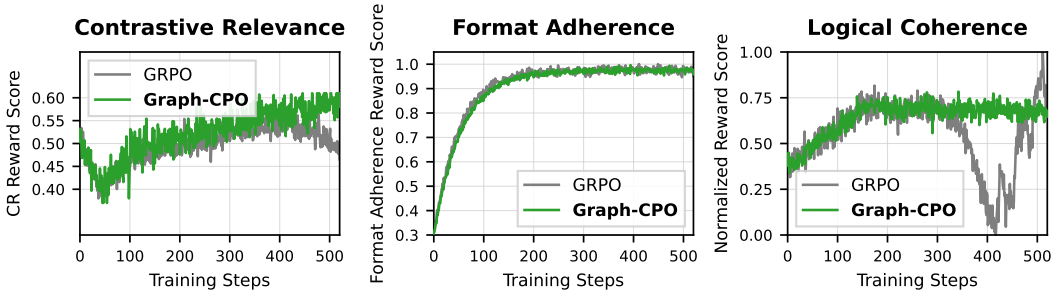


Figure 4: Training behavior of GRPO and Graph-CPO across Contrastive Relevance, Format Adherence, and Logical Coherence rewards on Qwen3-32B.

without R_{form} suggests that format control mainly stabilizes parsing and optimization rather than directly encoding mathematical reasoning. Finally, both GRPO scalar aggregation and the variant without dynamic weighting underperform full Graph-CPO, demonstrating that independently normalized rewards and curriculum-based weighting help prevent easy objectives from dominating harder pedagogical signals.

6.4 End-to-End Example: From Problem to Socratic Questions

We illustrate the complete Graph-CPO pipeline using a representative math word problem.

6.4.1 Input Problem.

> *Sarah has 24 cupcakes. She gives 8 cupcakes to her friend Tom and then bakes 15 more cupcakes. If she wants to pack all her cupcakes into boxes that hold 5 cupcakes each, how many boxes will she need?*

6.4.2 Phase 1: Graph Construction. The pipeline extracts typed nodes and relations, producing the dependency graph below.

```
{v1: "Sarah has 24", v3: "gives 8 to Tom"}
        --> v5: Op(-) --> v6: after giving
{v6, v7: "bakes 15 more"}
        --> v8: Op(+) --> v9: total cupcakes
{v9, v10: "boxes hold 5"}
        --> v11: Op(div) --> v13: goal (boxes needed)
{v12: condition: ceiling (round up)}
        --> v11
```

Figure 5: Semantic Dependency Graph of the running example expressed as prerequisite edges (prerequisite → dependent).

Key extracted nodes: v_1 (24 cupcakes), v_3 (8 given), v_5 (subtraction op), v_6 (intermediate result), v_7 (15 baked), v_8 (addition op), v_9 (total), v_{10} (box size: 5), v_{11} (division op), v_{13} (goal: boxes needed).

After 3 GNN layers, each node obtains a contextual embedding incorporating its prerequisites and downstream dependencies.

6.4.3 Phase 2: Warm-Start Initialization. The policy π_θ (Qwen3-32B) is initialized via supervised learning on annotated Socratic sequences, ensuring fluent question generation before RL optimization.

6.4.4 Phase 3: Graph-CPO Sequential Generation. Table 13 shows the step-by-step generation process:

Key mechanisms: **1. Frontier Tracking:** At each timestep, only nodes whose prerequisites are resolved enter the frontier. For example, v_5 (subtraction operation) becomes available only after both v_1 and v_3 are resolved.

2. Contrastive Relevance: At $t = 3$, the reward computation is:

$$R_{\text{rel}}(q_3, v_5) = \cos(\text{Enc}(q_3), \mathbf{h}_{v_5}) - 0.5 \times \max_{u \in \{v_1, v_3\}} \cos(\text{Enc}(q_3), \mathbf{h}_u)$$

This encourages alignment with the current target (v_5) while penalizing repetition of resolved nodes (v_1, v_3).

3. Curriculum Weighting: Early in training ($u \approx 25$), format adherence receives weight $\omega_{\text{form}} \approx 0.65$. By $u = 250$, this decays to $\omega_{\text{form}} \approx 0.16$, shifting emphasis toward relevance and coherence.

6.4.5 Comparison with Baseline Methods. **Observations:** Generic and keyword-based methods produce questions that are either too vague or violate dependency order. The Qwen3-32B baseline generates reasonable questions but skips the subtraction step, creating a logical gap. Graph-CPO follows the exact dependency topology, generating five targeted questions that scaffold the complete reasoning path.

Qualitative comparison with the baseline. Table 15 provides a representative side-by-side example. The MWP is: *Maria has 12 apples. She buys 5 more apples and then gives 4 apples to her brother. How many apples does Maria have now?* The extracted SDG path is: $12, 5 \rightarrow (+) \rightarrow$ subtotal; subtotal,$4 \rightarrow (-) \rightarrow$ goal.

Table 15: Qualitative comparison of generated Socratic sub-questions.

Step	Target node	Graph-CPO question	Baseline question
1	Initial quantity	How many apples does Maria have before any changes occur?	What is the story about?
2	Addition / subtotal	After Maria buys 5 more apples, what is her total before giving any away?	Can you add the numbers?
3	Subtraction / goal	After finding that subtotal, what calculation represents giving 4 apples to her brother?	What is the final answer?

Table 13: Graph-CPO generation trace showing frontier evolution and targeted questions.

Step	Resolved Nodes	Frontier	Generated Question
$t = 0$	$\emptyset$	$\{v_1, v_3, v_7, v_{10}\}$	–
$t = 1$	$\emptyset$	v_1 selected	*How many cupcakes does Sarah have at the start?*
$t = 2$	$\{v_1\}$	v_3 selected	*How many cupcakes does Sarah give to Tom?*
$t = 3$	$\{v_1, v_3\}$	v_5, v_6 selected	*After giving 8 to Tom, how many does Sarah have left?*
$t = 4$	$\{v_1, v_3, v_5, v_6\}$	v_7, v_8, v_9 selected	*After baking 15 more, what is Sarah's new total?*
$t = 5$	$\{v_1, \ldots, v_9\}$	v_{10}, v_{11}, v_{13} selected	*If each box holds 5 cupcakes, how many boxes will she need?*

Table 14: Question sequences generated by different methods for the same problem.

Method	Generated Questions
Generic QG	1. What is this problem about? 2. What operations do you see? 3. Can you solve it?
Keyword-based	1. How many cupcakes in total? [skips intermediate steps] 2. What does Sarah do? [too vague] 3. How many boxes? [violates prerequisite order]
Qwen3-32B baseline	1. How many cupcakes does Sarah start with? 2. How many after baking more? [skips subtraction step] 3. How do you pack them?
Graph-CPO (ours)	1. How many cupcakes does Sarah have at the start? 2. How many cupcakes does Sarah give to Tom? 3. After giving 8 to Tom, how many does Sarah have left? 4. After baking 15 more, what is Sarah's new total? 5. If each box holds 5, how many boxes will she need?

Graph-CPO follows the SDG topology by first identifying the initial quantity, then deriving the intermediate subtotal, and finally targeting the subtraction needed for the goal. In contrast, the baseline questions are more generic and less explicitly tied to the current reasoning frontier.

6.4.6 Reward Dynamics During Training. At training update $u = 250$, for the subtraction question (q_3), the multi-objective advantage computation proceeds as follows:

Given 8 rollouts with mean rewards $\mu_{\text{rel}} = 0.82$, $\mu_{\text{coh}} = 0.88$, $\mu_{\text{form}} = 1.0$ and standard deviations $\sigma_{\text{rel}} = 0.07$, $\sigma_{\text{coh}} = 0.05$, $\sigma_{\text{form}} = 0.0$, the advantage for a rollout with rewards $(0.88, 0.91, 1.0)$ is:

$$A_{\text{rel}} = \frac{0.88 - 0.82}{0.07} = 0.86 \tag{19}$$

$$A_{\text{coh}} = \frac{0.91 - 0.88}{0.05} = 0.60 \tag{20}$$

$$A_{\text{form}} = 0.0 \quad \text{(all rollouts have perfect format)} \tag{21}$$

With curriculum weights $\omega_{\text{form}}(250) = 0.16$ and $\omega_{\text{rel}}(250) = \omega_{\text{coh}}(250) = 0.42$:

$$\hat{A} = 0.42 \times 0.86 + 0.42 \times 0.60 + 0.16 \times 0.0 = 0.61$$

This positive advantage increases the probability of generating similar questions. Critically, independent normalization prevents the saturated format reward from dominating the learning signal.

7 Limitations

While Graph-CPO demonstrates strong performance, several limitations warrant discussion. First, the Semantic Dependency Graph construction pipeline achieves 94.3% critical path preservation but still introduces some structural errors, particularly for problems with implicit operations or complex multi-hop reasoning. Second, our approach assumes that effective pedagogical scaffolding follows the topological structure of mathematical dependencies, which may not capture all aspects of human learning, such as conceptual difficulty or metacognitive considerations. Third, computational cost remains higher than supervised baselines due to rollout sampling during RL training, though this overhead is modest compared to scalar GRPO (636 vs 610 GPU-hours). Finally, our evaluation focuses on mathematical reasoning; the generalizability to other domains requiring structured scaffolding (e.g., physics problem-solving, logical argumentation) remains to be investigated.